

(1) Over-flow Detection in signed 8-bit 2's complement Addition

Debug process:

1) In 8-bit 2's complement, range = -128 to +127

2) Overflow occurs when

* Two positive numbers produce a negative result

* Two negative numbers produce a positive result

Example:

$$\begin{array}{r} 01100100 (+100) \\ 00111100 (+60) \\ \hline 10100000 (-96) \end{array}$$

Actual result should be 160, which is outside the 8 bit range.

Hence overflow has occurred.

Overflow Detection

Overflow Handling:

- 1) Generate an Overflow Flag (V).
- 2) Use saturation arithmetic (limit result to +127 or -128)
- 3) Generate an exception or interrupt if required.

Result:

Reliable signed arithmetic with correct overflow detection.

2) Ripple Carry Adder Delay and Carry Look-Ahead Adder.

Cause of Delay:

In a Ripple Carry Adder (RCA), each full adder waits for

the previous carry.

Debug observation:

Delay increases with number of bits

Worst-case delay occurs when carry propagates through every stage.

Solution: Carry Look-Ahead Adder (CLA).

Generate:

$$\text{Generate } (G) = A \times B$$

$$\text{Propagate } (P) = A \oplus B$$

Carry equations:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

Advantages:

- 1) Faster addition.
- 2) Lower propagation delay.
- 3) Better processor performance.
- 4) Suitable for high-speed arithmetic units.

3) Booth's Algorithm, Shifting

Check Binary Shifting

For 2^n reduction

2. Addition / subtraction operation

3. Arithmetic right shift

4. Sign extension.

Booth's Rules:

Bit	Q_n	Next bit
0	1	No operation
1	0	$A = A - B$
0	0	$A = A + B$
1	1	No operation

Common faults:

- 1) Wrong bit-pair evaluation.
- 2) Logical shift used instead of arithmetic shift.
- 3) Incorrect sign extension
- 4) Missing final shift.

Modifications:

- 1) Use correct Both decision table.
- 2) Perform arithmetic right shift.
- 3) Preserve sign bit during shifting
- 4) Verify sign extension after every iteration

Result:

Correct multiplication for positive and negative numbers.

4) Restoring vs Non-Restoring Division.

Problem:

In restoring division:

After subtraction,

If remainder becomes negative

This extra restoration increases execution time

Restoring Division.

Subtract divisor $\rightarrow$ Negative? $\rightarrow$ Yes $\rightarrow$ Restore number.

Extra restoration operations reduce speed.

Solution: Non-Restoring Division

Instead of restoring

- 1) If remainder is positive $\rightarrow$ Subtract divisor
- 2) If remainder is negative $\rightarrow$ add divisor in next iteration

Advantages:

- 1) Fewer arithmetic operations.
- 2) Faster execution.
- 3) Less hardware complexity.
- 4) Improved processor performance.

5) IEEE 754 Floating Point Accuracy.

Debug steps:

1) Exponent Alignment

Shift the mantissa of the smaller number until exponents match.

2) Addition / Subtraction

Perform operation on aligned mantissas

3) Normalisation

Adjust mantissa so it begins with 1.xxxx.

4) Rounding

Apply IEEE 754 rounding.

cause of inaccuracy:

When numbers have very different magnitudes

Optimization Techniques:

1) Use double precision (64-bit) instead of single precision

2) Use higher precision arithmetic

3) Apply numerically stable algorithms (e.g., Kahan summation).

4) Reduce repeated rounding errors.

Results:

Improved floating-point accuracy and reliable numerical computation.